



Yggdrasil ERP — Security Architecture

Mimir Labs Technical Publication

Document:	ML-WP-007	Version:	1.2
Author:	Christopher Gaither	Date:	April 2026
Classification:	Public		

0.1 Executive Summary

Security is not a bolt-on feature of Yggdrasil ERP — it is woven into every layer of the stack, from the C++ server's memory-safe patterns to the PostgreSQL database's row-level security policies. This white paper provides a comprehensive overview of the security architecture, covering authentication, authorization, encryption, multi-tenancy isolation, audit logging, infrastructure hardening, and compliance posture. Yggdrasil has completed its SOC 2 Type I readiness assessment and maintains 10 formal security policy documents.

0.2 1. Security Philosophy

Yggdrasil's security architecture is built on five principles:

1. **Least privilege** — Users, services, and processes receive only the minimum access required for their function.
2. **Defense in depth** — No single control is relied upon. Overlapping controls at network, application, and data layers provide resilience.
3. **Tenant isolation** — Multi-tenant data is logically separated at the database level and must never be exposed across tenant boundaries.
4. **Fail secure** — When a control fails, the system defaults to a denied/restricted state.
5. **Auditability** — All security-relevant actions are logged with sufficient detail to reconstruct what happened, when, and by whom.



0.3 2. Authentication

0.3.1 2.1 Password Security

Yggdrasil uses PBKDF2-HMAC-SHA256 for password hashing, a NIST-approved key derivation function:

Parameter	Value
Algorithm	PBKDF2-HMAC-SHA256
Iterations	600,000
Salt length	32 bytes (cryptographically random)
Hash output	256 bits
Comparison	Constant-time (prevents timing attacks)

The iteration count follows OWASP 2023 recommendations and exceeds the NIST SP 800-132 minimum. Each user has a unique random salt, preventing rainbow table attacks even if the hash database is compromised.

Password complexity requirements: - Minimum length enforcement - Mixed case (uppercase and lowercase) - Numeric digits required - Special characters required - Password history — prevents reuse of last N passwords

0.3.2 2.2 Multi-Factor Authentication (MFA)

TOTP (Time-based One-Time Password) per RFC 6238:

Parameter	Value
Algorithm	HMAC-SHA1 (per RFC 6238 specification)
Time step	30 seconds
Code length	6 digits
Clock skew tolerance	± 1 time step (90-second window)
Recovery codes	10 single-use codes generated on enrollment

MFA enrollment flow: 1. User requests MFA enrollment via API 2. Server generates a cryptographically random secret key 3. Secret is returned as a `otpauth://` URI (scannable by authenticator apps) 4. User confirms enrollment by providing a valid TOTP code 5. 10 recovery codes are generated and returned (one-time display) 6. Subsequent logins require email + password + TOTP code

0.3.3 2.3 Account Lockout

Progressive lockout policy to prevent brute-force attacks:



Tier	Lockout Duration	Trigger
1	15 minutes	5 failed attempts
2	30 minutes	5 more failed attempts
3	1 hour	5 more failed attempts
4	4 hours	5 more failed attempts
5	24 hours	5 more failed attempts

- Lockout tier escalates with repeated failures
- Successful login resets the failure counter
- Administrators can manually unlock accounts via the admin API
- Lockout events are logged with client IP for incident investigation

0.3.4 2.4 Session Management

Token	Lifetime	Storage	Transport
Access JWT	1 hour	HttpOnly cookie	Secure; SameSite=Lax
Refresh JWT	7 days	HttpOnly cookie	Secure; SameSite=Lax; Path=/api/auth

Cookie security flags: - HttpOnly — Prevents JavaScript access, mitigating XSS token theft - Secure — Cookies transmitted only over HTTPS - SameSite=Lax — Prevents CSRF on state-changing requests while allowing navigation

JWT payload contents: user ID, tenant ID, role, module permissions, issued-at timestamp, expiration. The signing key (HMAC-SHA256) is loaded from configuration and supports quarterly rotation.

0.4 3. Authorization

0.4.1 3.1 Role-Based Access Control (RBAC)

Five-tier role hierarchy:

Role	Scope	Capabilities
DevAdmin	System-wide	All operations, cross-tenant access, infrastructure management
TenantAdmin	Single tenant	Full access within tenant, user management, configuration



Role	Scope	Capabilities
Manager	Module-level	CRUD + approval authority within assigned modules
User	Module-level	Standard CRUD within assigned modules
ReadOnly	Module-level	View-only access within assigned modules

0.4.2 3.2 RBAC Middleware

The `enforceRbac()` middleware intercepts every API request:

1. **Module auto-detection** — Determines the target module from the request path and database table name
2. **Permission resolution** — Looks up the user's role and module-level permissions from the JWT
3. **Authorization decision** — Grants or denies based on the operation type (read/write/admin) and role
4. **Fail-closed default** — Routes not explicitly mapped to a module are denied. This prevents newly added endpoints from accidentally being accessible without authorization

0.4.3 3.3 Admin-Gated Routes

Administrative endpoints (user management, system configuration, data management, backup/restore) are additionally gated behind an admin role check that verifies `isAdmin` status from the `AuthManager`. This provides a second layer of defense beyond RBAC module permissions.

0.4.4 3.4 Governed State Transitions (ROPE)

RBAC answers *who may call an endpoint*. Runtime Operational Policy Enforcement (ROPE) answers *which state changes are permitted at all*. The State Constraint Engine enforces governed state transitions **inside the database write transaction** and refuses illegal ones at the gate. Because the policy check lives in the transaction path — not in a model wrapper or an application layer that could be bypassed — there is no code path that commits a disallowed transition. ROPE enforces per-tenant: each tenant's operational policy governs its own writes.

This matters most for autonomous and AI-driven workflows. External LLM agents reach the system only through a read/propose-only agent-tools surface; they can read data and propose actions, but the substrate decides what is actually allowed. An agent cannot bypass the enforcement gate, because the gate is the write transaction itself.

0.5 4. Multi-Tenant Isolation



0.5.1 4.1 Database-Level Isolation

Every tenant-scoped table (120+ tables) includes a `tenant_id` UUID column. Isolation is enforced at two levels:

Application-level enforcement: - Every SQL query that touches tenant data includes a `WHERE tenant_id = ?` clause - The `RouteHelpers::resolveTenantId()` function extracts the tenant ID from the authenticated JWT — it is never taken from user-supplied query parameters - The `QueryBuilder` automatically injects tenant filters

Database-level enforcement (Row-Level Security): - PostgreSQL RLS policies are defined on all tenant-scoped tables, with `FORCE ROW LEVEL SECURITY` so the policies apply even to the table owner - Policies reference `current_setting('app.current_tenant_id')`, a session variable set at the start of each database transaction - Even if application code fails to filter by tenant, the database rejects cross-tenant access - This defense-in-depth approach means a single application bug cannot leak tenant data

0.5.2 4.2 Tenant Context Flow

1. Client sends request with JWT cookie
2. Server extracts `tenant_id` from JWT payload
3. Server sets PostgreSQL session variable: `SET app.current_tenant_id = '<uuid>'`
4. All queries execute with RLS policies active
5. RLS policies compare row `tenant_id` against session variable
6. Non-matching rows are invisible – `SELECTs` return nothing, `INSERTs` fail

0.5.3 4.3 Cross-Tenant Protection

- No API endpoint accepts a tenant ID from request parameters — it is always derived from the authenticated session
- B2B Event Hub scopes WebSocket channels per tenant — cross-tenant event leakage is architecturally impossible
- The DevAdmin role can operate across tenants for system administration, but this capability is logged and auditable

0.6 5. Encryption

0.6.1 5.1 Data in Transit



Path	Encryption
Client → Server	TLS 1.2+ via Cloudflare Zero Trust tunnel
WebSocket connections	WSS (TLS) via tunnel or server-side SSL
Server → Redpanda	TLS via Cloudflare tunnel
Server → PostgreSQL	Internal Docker network (not externally accessible)

Prohibited protocols: TLS 1.0, TLS 1.1, SSL 3.0, unencrypted HTTP on production endpoints.

0.6.2 5.2 Data at Rest

Data Type	Protection
Passwords	PBKDF2-HMAC-SHA256 (600k iterations, 32-byte salt)
Sensitive configuration	AES-256-GCM via SecretsCrypto service
Database volumes	Filesystem-level encryption (LUKS) on production hosts
Backups	AES-256 encryption + SHA-256 integrity checksums

0.6.3 5.3 Cryptographic Standards

Use Case	Algorithm	Key Size
TLS certificates	RSA 2048-bit or ECDSA P-256	2048+ bit
JWT signing	HMAC-SHA256	256-bit
Password hashing	PBKDF2-HMAC-SHA256	256-bit output
Symmetric encryption	AES-256-GCM	256-bit
Backup encryption	AES-256	256-bit
Integrity verification	SHA-256	256-bit

Prohibited algorithms: MD5, SHA-1 (for security purposes), DES, 3DES, RC4, RSA < 2048-bit, raw SHA-256 for password hashing.

0.6.4 5.4 Key Management

- Production keys stored in a designated secrets vault, separate from application code
- Each environment (development, staging, production) uses distinct keys
- JWT signing key rotated quarterly with a 24-hour grace window for token expiry
- Database credentials rotated annually or on personnel change



- Keys are never committed to source control

0.7 6. Input Validation and Injection Prevention

0.7.1 6.1 SQL Injection Prevention

Multiple layers protect against SQL injection:

1. **Prepared statements** — All database queries use Qt's `QSqlQuery::prepare()` with parameterized `addBindValue()`. User input never appears in SQL strings.
2. **QueryBuilder escaping** — The `escape()` and `escapeLike()` methods provide additional protection for dynamic query construction.
3. **Data DMZ** — The OpenAPI specification enforces `additionalProperties: false` on all create/update schemas, rejecting unknown fields at the API boundary. 1,404 format constraints and 878 `maxLength` limits prevent oversized or malformed input from reaching the database.
4. **StrictJsonParser** — No type coercion (string→number, empty→null UUID). Type mismatches produce immediate rejection errors.

0.7.2 6.2 UUID Validation

Tenant IDs and entity IDs are validated as proper UUIDs before use in queries. The server rejects malformed UUIDs at the route handler level, preventing injection through ID parameters.

0.7.3 6.3 Path Traversal Prevention

File upload and attachment endpoints sanitize paths to prevent directory traversal attacks. File names are normalized to remove `../` sequences and validated against an allowlist of permitted characters.

0.7.4 6.4 XSS Prevention

- Server responses set appropriate `Content-Type` headers
- JSON responses are serialized through Qt's `QJsonDocument`, which escapes special characters
- The web application uses React's built-in XSS protection (automatic escaping of rendered values)

0.8 7. Audit Trail



0.8.1 7.1 Audit Change Log

Every data mutation is recorded in the `audit_change_log` table:

Column	Content
<code>id</code>	UUID primary key
<code>tenant_id</code>	Tenant context
<code>user_id</code>	Authenticated user who performed the action
<code>action</code>	Operation type: INSERT, UPDATE, DELETE, STATUS_CHG
<code>table_name</code>	Target table
<code>entity_id</code>	Target entity UUID
<code>old_values</code>	JSONB of previous values (for UPDATE/DELETE)
<code>new_values</code>	JSONB of new values (for INSERT/UPDATE)
<code>changed_at</code>	Timestamp of the change

0.8.2 7.2 State Machine Audit

All status transitions through the StateMachine service are logged with `action = 'STATUS_CHG'` and JSONB before/after state values. Partial indexes on (`table_name`, `entity_id`, `changed_at` DESC) WHERE `action = 'STATUS_CHG'` enable efficient status history queries.

0.8.3 7.3 Immutable Audit Records

The audit trail is tamper-evident at the database level. Both the `audit_change_log` and `ledger_entries` tables carry database triggers that reject UPDATE, DELETE, and TRUNCATE operations. Once a change or ledger entry is written, it cannot be altered or removed, even by a privileged database session. This makes the audit history append-only by enforcement rather than by convention, and means any attempt to rewrite history is blocked at the gate rather than detected after the fact.

0.8.4 7.4 Security Event Logging

The structured JSON logger records security-relevant events:

- Authentication attempts (success and failure, with client IP)
- Authorization decisions (grants and denials)
- Account lockouts and unlocks
- MFA enrollment and verification events
- Administrative actions (user creation, role changes, configuration changes)
- Password changes and resets

0.8.5 7.5 Log Protection

- Log files use automatic rotation (10 MB per file, 10 files retained)



- PII redaction is applied to log entries — sensitive fields are masked before writing
- Log files are stored outside the web root with restricted file permissions

0.9 8. Infrastructure Security

0.9.1 8.1 Network Architecture

The production deployment uses a Cloudflare Zero Trust tunnel architecture:

Internet → Cloudflare Edge → Encrypted Tunnel → VPS → Docker Network → Services

- **No direct internet exposure** — The VPS firewall (ufw) defaults to deny-all inbound. Only SSH (port 22) is open externally.
- **Application ports** (HTTP 8080, WebSocket 8081, Redpanda 19092) are accessible only through Cloudflare tunnels.
- **Docker bridge networking** — Inter-container communication uses an isolated network with no published ports beyond what the Compose stack requires.

0.9.2 8.2 Host Hardening

- fail2ban monitors SSH for brute-force attempts (5 retries, 1-hour ban)
- No password-based SSH access to production — key-based authentication only
- System provisioned via `infra/vps/provision.sh` with OS hardening at first boot
- Automatic security updates enabled
- Single non-root service account (`yggdrasil`) for all application processes

0.9.3 8.3 Backup Security

- Daily `pg_dump` backups encrypted with AES-256 before writing to disk
- Encryption keys stored separately from backup storage
- SHA-256 checksums generated for integrity verification
- Backup restoration requires cross-tenant guard validation

0.10 9. Compliance Posture

0.10.1 9.1 SOC 2 Type I Readiness

Yggdrasil has completed a SOC 2 Type I readiness assessment covering all five Trust Service Criteria:



Criterion	Status
Security	Controls implemented
Availability	Controls implemented
Processing Integrity	Controls implemented
Confidentiality	Controls implemented
Privacy	Controls implemented

0.10.2 9.2 Policy Framework

10 formal security policy documents govern the security program:

1. **Information Security Policy** — Master policy establishing the security program
2. **Access Control Policy** — User provisioning, least privilege, quarterly reviews
3. **Encryption Policy** — In-transit, at-rest, and key management requirements
4. **Data Classification Policy** — Public, Internal, Confidential, Restricted tiers
5. **Data Retention & Disposal Policy** — Retention periods and secure destruction
6. **Incident Response Plan** — Detection, triage, containment, and post-mortem procedures
7. **Business Continuity / DR Plan** — RTO/RPO targets, failover, communication
8. **Change Management Policy** — Code review, CI gates, rollback procedures
9. **Vendor Risk Management Policy** — Third-party security assessments
10. **Privacy Policy** — PII handling, GDPR/CCPA compliance

0.10.3 9.3 Vulnerability Management

Severity	Remediation SLA
Critical	72 hours
High	7 calendar days
Medium	30 calendar days
Low	Next scheduled release

- Dependency scanning runs in CI on every pull request
- Static analysis integrated into the build pipeline
- Third-party penetration testing conducted annually

0.10.4 9.4 Known Gaps and Remediation

Gap	Risk	Target
Direct TLS not yet enabled	HIGH	Q2 2026
Account lockout is in-memory	LOW	Q2 2026



Gap	Risk	Target
Code review not enforced via branch protection	MEDIUM	Q2 2026

These gaps are documented and tracked with specific remediation targets.

0.11 10. Security Development Lifecycle

0.11.1 10.1 Secure Coding Practices

- **C++17 memory safety** — RAII patterns, smart pointers, bounds-checked containers. No raw `new/delete` in application code.
- **Qt framework guarantees** — Qt's string handling, JSON parsing, and SQL driver provide built-in protection against buffer overflows and format string attacks.
- **Compiler hardening** — `-Wall -Wextra -Wpedantic` warnings treated as guidance; security-relevant warnings are resolved before merge.

0.11.2 10.2 CI/CD Security Gates

Gate	Purpose
ESLint	Static analysis for web application JavaScript/TypeScript
TypeScript strict mode	Type safety enforcement
Vitest	Unit test suite
Catch2	C++ unit test suite
Schema validation	PostgreSQL schema + migration + seed data verification
npm audit	Dependency vulnerability scanning

0.11.3 10.3 Incident Response

The incident response plan defines a structured process:

1. **Detection** — Automated monitoring, user reports, or external notification
2. **Triage** — Severity classification (Critical/High/Medium/Low)
3. **Containment** — Isolate affected systems, preserve evidence
4. **Eradication** — Remove root cause, patch vulnerabilities
5. **Recovery** — Restore services, verify integrity
6. **Post-mortem** — Document findings, update controls, communicate to stakeholders

Tenant notification is required when Confidential data may have been exposed.



0.12 11. Summary of Security Controls

Layer	Controls
Network	Cloudflare tunnel, ufw deny-all, fail2ban, no direct port exposure
Transport	TLS 1.2+, WSS, HTTPS-only cookies
Authentication	PBKDF2 (600k iter), TOTP MFA, progressive logout, password history
Authorization	RBAC middleware, fail-closed, admin gating, ROPE governed state transitions
Data	FORCE RLS policies, tenant_id on all tables, AES-256-GCM at rest
Application	Prepared statements, input validation, Data DMZ, no type coercion
Audit	Immutable change log + ledger (tamper-evident triggers), status transition log, structured JSON logging, PII redaction
Operations	Encrypted backups, automated health checks, security policy framework

Copyright 2026 Mimir Labs. All rights reserved.